

SYSC4805

Computer Systems Design Lab

Autonomous Snowplow Final Report



Group L3-G15 Infinitely Hot [1]

Karran Dhillon, 101229275

Alex Tempel, 101225006

Collin Dang, 101233892

Qasim Hamid, 101237762

TA: Igor Bogdanov, Mahya Shahmohammadimehrjardi

Table of Contents

1	Project Charter	3
1.1	Project Objective.....	3
1.2	Overall Deliverables	3
2	Scope	4
2.1	Functional Requirements	4
2.2	Non-Functional Requirements.....	4
2.3	Constraints	4
2.4	Detailed Deliverables	5
2.5	Testing Plan.....	5
2.5.1	Success Criteria	6
2.5.2	Unit Testing	6
2.5.3	Integration Testing	7
3	Schedule	7
3.1	Activities	7
3.2	Schedule Network Diagram	9
3.3	Gantt Chart.....	9
4	Cost.....	10
5	Human Resources.....	11
5.1	Responsibility Assignment Matrix.....	11
5.2	Contribution of Each Member	13
6	Progress Update	14
6.1	System Architecture.....	14
6.2	Sequence Diagram	16
6.3	Watchdog Timer Implementation	17
6.4	Testing Update	18
6.4.1	Unit Test Results.....	18
6.4.2	System Integration Testing Results	21
6.4.3	Control Chart Results	22
6.5	State chart.....	24
6.6	Updated Planned Value Analysis	25
6.7	Software Development	26
6.8	RTOS Implementation.....	27
7	References.....	29

1 Project Charter

1.1 Project Objective

This project presents an Autonomous Snowplow designed to clear snow off an enclosed area without hitting obstacles in its path. This project contains constraints on time, the speed of the robot, and pre-defined boundaries to ensure the system can autonomously adjust to different sized objects. The project also involves designing a complex driving algorithm to account for objects along its path.

Our team, Infinitely Hot, composed of Carleton Computer Systems Engineering Students aims to showcase our solution to the design of the Autonomous Snowplow and it's impacts on snow removal. This document outlines the system's key features, technical components, and our implementation plan. The report also lists key milestones, timelines, and deliverables to succeed in creating an autonomous robot to clear snow.

By the end of the report, the team will clearly demonstrate the systems design and the progress made in the design and implementation of the Autonomous Snowplow.

1.2 Overall Deliverables

The following list describes the necessary milestone deliverables that will show the abilities of the Autonomous Snowplow.

Table 1: Overall deliverables for the project

ID	Major Deliverables	Description
D1	Project proposal	Lays out the fundamental scope, timeline, and costs associated with the project
D2	Progress Report	Discusses how the development and design is progressing as the project meets its middle mark in terms of timeline.
D3	Oral Presentation	Present to the class how the vehicle is driving, the backing algorithm, and how it was designed.
D4	The vehicle shall drive within the boundaries while responding to dynamic obstacles in the path	The vehicle must create a path based on what objects already exist; this is used to physically plow the snow while responding to objects that are moving.
D5	Final Demo	The TAs will setup a random, enclosed test area which the vehicle must be able to plow the snow within. It's a culmination of D4 and D5 that allow success in this final deliverable.

For more in-depth deliverables, please see the WBS provided in the Section 2.

2 Scope

2.1 Functional Requirements

As part of the project, provided below are the functional requirements describing the features, behaviours, and functions of the autonomous robot. The requirements are dissected from the project description [2].

Table 2: Functional requirements of the project

ID	Requirement
FR1	The vehicle shall push wooden cubes outside the enclosed rectangular area
FR2	The vehicle shall detect crossing the rectangular boundary with black painter's tape
FR3	The vehicle shall avoid dynamic obstacles along its plowing path

2.2 Non-Functional Requirements

Similar to the functional requirements, the following table describes additional, non-functional requirements, it must support.

Table 3: Non-functional requirements of the project

ID	Requirement
NFR1	The vehicle shall push more than 95% of the wooden cubes in the enclosed rectangular area
NFR2	The vehicle shall include a unique plow attachment that can push 20 wooden cubes concurrently
NFR3	The vehicle shall finish moving all the wooden cubes within 5 minutes

2.3 Constraints

In terms of the restrictions placed on the system and how it can be built, they are provided in the following table as per the project description [2].

Table 4: Constraints for the system

ID	Requirement
C1	The vehicle shall not exceed 30cm/s in speed
C2	The vehicle's dimensions shall not exceed 226 x 262 x 150 (LxWxH) mm
C3	The vehicle shall only start from a corner inside the perimeter and parallel to the perimeter
C4	The vehicle shall not be touched during operation

2.4 Detailed Deliverables

Provided below is the WBS for the system. The WBS describes the overall work packages and therefore, deliverables in order to be successful in the development of the project.

The WBS has been updated since the project proposal to better reflect the final iteration of the project. The primary parts removed since the proposal is the mapping algorithm, IMU sensor, and IR sensor. Removal of the mapping algorithm will be discussed in section 6, and the IMU/IR sensors were not needed in the design of the system. Note, they are still shown in the WBS due to the fact that function handlers were added for future iterations of the project, the team could integrate them into the system.

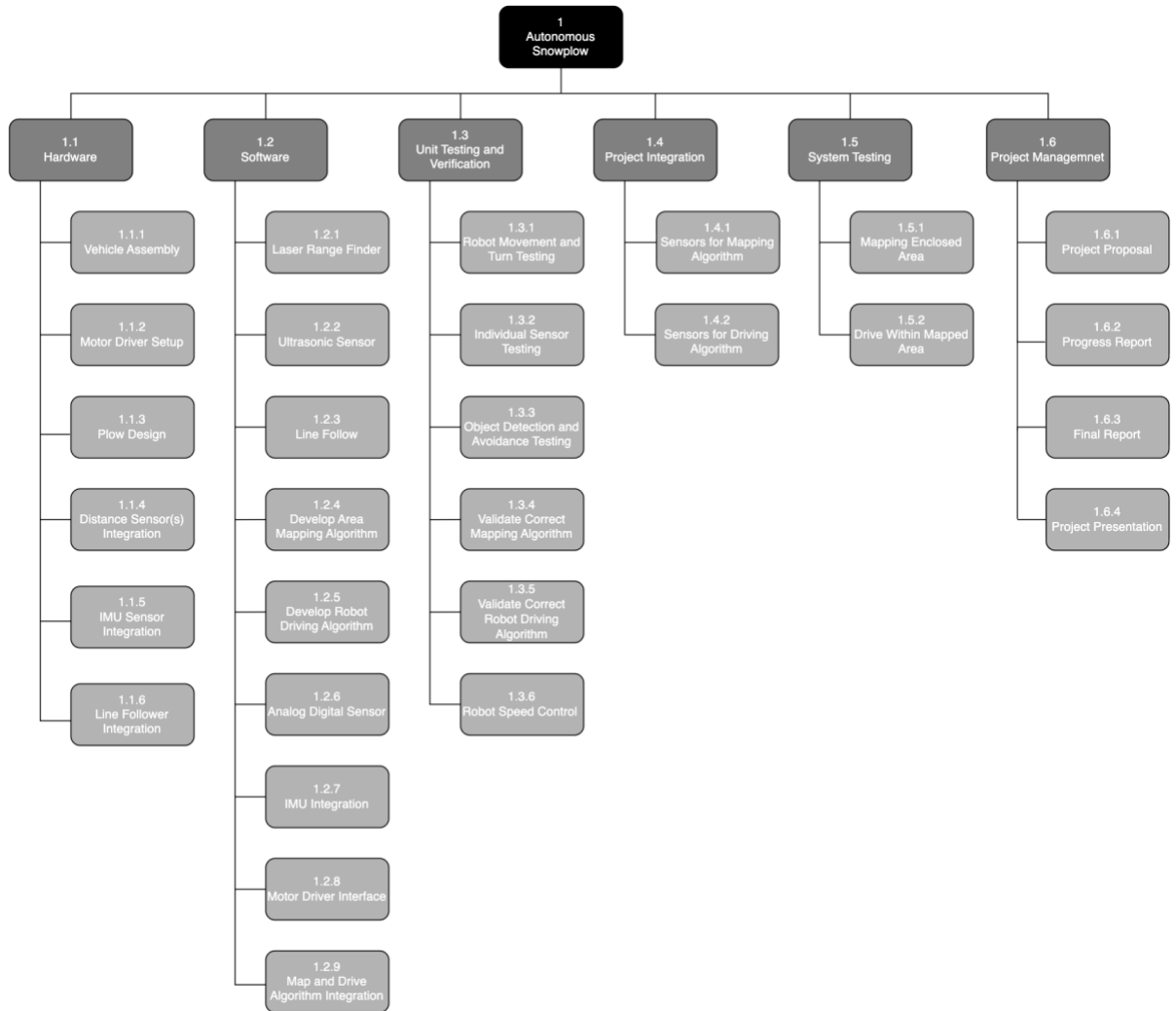


Figure 1: Work Breakdown Structure (WBS) for the project

2.5 Testing Plan

The following section describes how each part of the package, including unit testing, integration testing, and system testing is performed to ensure the system is ready to run during the demo.

2.5.1 Success Criteria

In order to define a successful project, outside of the functional and non-functional requirements being met, the system defines the following for completion:

1. The robot stays within the 2.5m x 2.5m area while plowing the snow
2. Moving objects are avoided in real-time
3. Snow is plowed and cleared from the area

2.5.2 Unit Testing

The following table describes the unit tests for each module that will be performed to verify base functionality, namely in the sensors and their readings. Unit testing is done, please see section 6 for more information.

Table 5: Unit tests for each module of the system

Activity Code	Test	Pass Condition
UT1	Laser range finder minimum response distance	Laser range finder reports an object 50cm away with ± 5 cm of error
UT2	Laser range finder maximum response distance	Laser range finder reports an object 200cm away with ± 5 cm of error
UT3	Laser range finder minimum rate of readings	Laser range finder records at least 3 value per second
UT3	Ultrasonic sensor maximum response distance	Ultrasonic sensor tracks objects 30cm away within ± 2 cm of error
UT4	Ultrasonic sensor minimum response distance	Ultrasonic sensor tracks objects 5cm away within ± 1 cm of error
UT5	Ultrasonic sensor rate of readings	Ultrasonic sensor records at least 3 values per second
UT6	Car minimum movement speed	The car can operate above 5cm/s
UT7	Line follower on with electric tape	The line follower reports a line when above an electric tape
UT8	Line follower off above ground	The line follower reports no line when above the ground
UT9	IR sensor detects object	The IR sensors report an object that is 5 cm away from it within ± 1 cm of error
UT10	IR sensor reports no object	The IR sensor should not report an item when an object is 100cm away from it
UT11	Analog Digital sensor close range detection	Analog digital sensor can report items 3cm away from it within ± 1 cm of error
UT12	Analog digital sensor max range detection	Analog digital sensor reads items 10cm away from it within ± 1 cm of error
UT13	IMU x,y,z-axis orientation accuracy	IMU accurately reports angle of all axes within 20 degrees of error
UT14	Encoder accurately reports speed	The encoder can track speed within 3cm/s
UT15	Plow can move 20 objects concurrently	Pushing the plow with our hand, at least 20 cubes (snow) should be moved at the same time

2.5.3 Integration Testing

The following table describes how the upper-level modules interact with one another including how the sensors will interface with the robot.

Table 6: Integration tests for the system

Activity Code	Test	Pass Condition
IT1	Ultrasonic sensor interfacing with moving robot	Ultrasonic sensor updates the proximity of the robot to an object while moving towards it, sending a signal when 10cm away within ± 2 cm of error
IT2	Robot can follow a line with the line follower	The robot can move around a rectangle's outline, staying within the bounds of the square.
IT3	Vehicle can traverse the area of the cube within 3 minutes	Assuming no objects in the way, the vehicle should navigate the area of the cube within 3 minutes with all components attached

Integration and system testing is done, please see section 6, progress update for the results of the testing.

3 Schedule

The following section describes the overall project timelines and the distribution of work between team members.

3.1 Activities

Provided below is a list of activities necessary to achieve the desired deliverables of the project. These match the deliverables discussed in the WBS above.

Even though the IR/IMU sensors were not used, function handlers were made for them in case future iterations of the project, the team could integrate them into the system.

Table 7: Activities performed during development of the robot

Activity Code	Activity	Associated Work Package	Description
1	Motor Driver Interface	1.1.2, 1.2.8, 1.1.4	Create functions for controlling the motors (turn to direction, go forward, go backward, etc)
2	Line Follower Handler	1.2.3	Create a handler for data from the Line Follower Sensor, to convert the raw values into usable numbers (distance)
3	IR Sensor Handler	1.2.1, 1.1.4	Create a handler for data from the IR Sensor, to convert the raw values into usable numbers (distance)
4	Ultrasonic Sensor Handler	1.2.2, 1.1.4	Create a handler for data from the Ultrasonic Sensor, to convert the raw values into usable numbers (distance)

Activity Code	Activity	Associated Work Package	Description
5	Analog Distance Sensor Handler	1.2.6, 1.1.4	Create a handler for data from the Analog Distance Sensor, to convert the raw values into usable numbers (distance)
6	IMU Handler	1.2.7	Create a handler for data from the IMU Sensor, to convert the raw values into usable numbers (direction, speed, etc)
7	Wheel Encoder Handler	1.2.5, 1.3.6	Create a handler for data from the Wheel Encoder Sensor, to convert the raw values into usable numbers (speed)
8	Obstacle Detection Algorithm	1.2.4, 1.2.5	Create an algorithm that based on the sensor data can detect if there's an obstacle nearby
9	Driving Algorithm	1.2.5	Create an algorithm for where to drive that can avoid obstacles and cover the whole arena
10	Plow Design	1.1.3	Design, obtain, and install an optimal plow for our vehicle
11	Vehicle Assembly	1.1.1	Install sensors at the optimal positions in the vehicle for the sensors for our application
12	Sensor Integration	1.4.1, 1.4.2, 1.2.4, 1.2.5	Combine data from multiple sensors into a single value (combine multiple distance sensors that point in the same direction)
13	Motor and Sensor Integration	1.1.2	Have sensors update as the vehicle moves, have vehicle movement regulated by sensor readings (do not exceed maximum speed, make sure turn points you in the right direction)
14	Sensor and Motor Testing	1.3.2, 1.3.1, 1.3.3	Test that the sensor handlers work correctly independently. Test that the motor interface moves the vehicle by the requested values.
15	Integration Testing	1.4.1, 1.4.2	Test that the sensor's readings can be used for the higher-level algorithms
16	System Testing	1.5.1	Test that the vehicle can drive effectively
17	Acceptance Testing	1.5.2	Test that the vehicle can push snow in an environment resembling the project demo
18	Project Proposal	1.6.1	Document outlining what the project will do, and how we'll do it
19	Progress Report	1.6.2	Document of how the project is doing, midway through
20	Project Presentation	1.6.4	Present the results of the project
21	Final Report	1.6.3	Document the results of the project

Note, that each peripheral has an associated unit test as part of the activity.

3.2 Schedule Network Diagram

The following diagram describes the dependencies between the activities in 3.1 and therefore, when the team can start development on each.

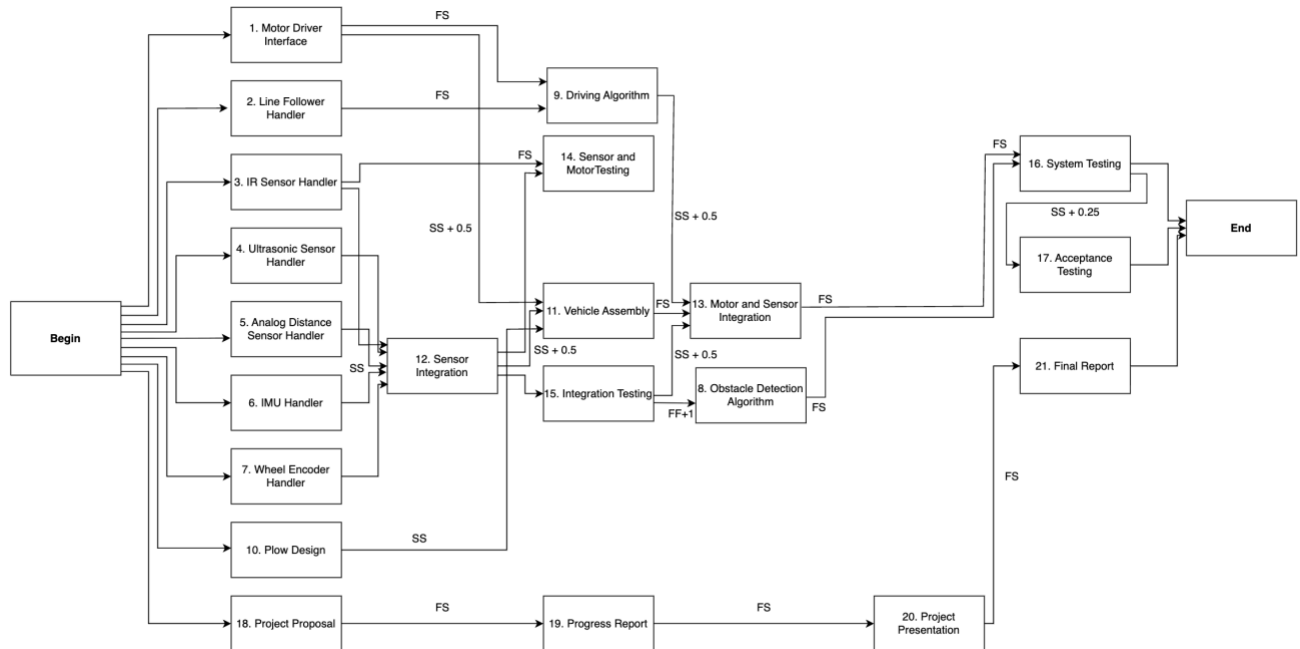


Figure 2: Schedule Network Diagram (SND) showing dependencies of activities in project

As the final parts of the project begin to wrap up, the team is on the final activity, the final report. Acceptance and system testing were performed during the demo lab. Please see section 6 for more information.

3.3 Gantt Chart

Provided below, is the organization, in time, of the activities described in 3.1. Note, there are two columns in each lab so activities can finish in half a lab. The vehicle assembly occurs twice: for initial motor testing and the second for after unit testing when each sensor is done. Milestones are listed as black diamonds with white diamonds as milestones accomplished.

The Gantt chart has been updated based on the team's final timeline, the time actually taken for the activity, and milestones hit.

In terms of major changes made: the individual sensor interfaces took the entire lab 6 (1.5 weeks), the driving and obstacle detection algorithms only took 1.5 weeks rather than the estimated 2 weeks (this was due to concrete, easy to implement sensor functions). Plow design is now cardboard rather than 3D print, so the time was delayed to later in the project (second half of lab 8). Vehicle assembly was needed for the algorithms and thus, done earlier. Sensor integration only took 1 lab.

AUTONOMOUS SNOWPLOW FINAL REPORT

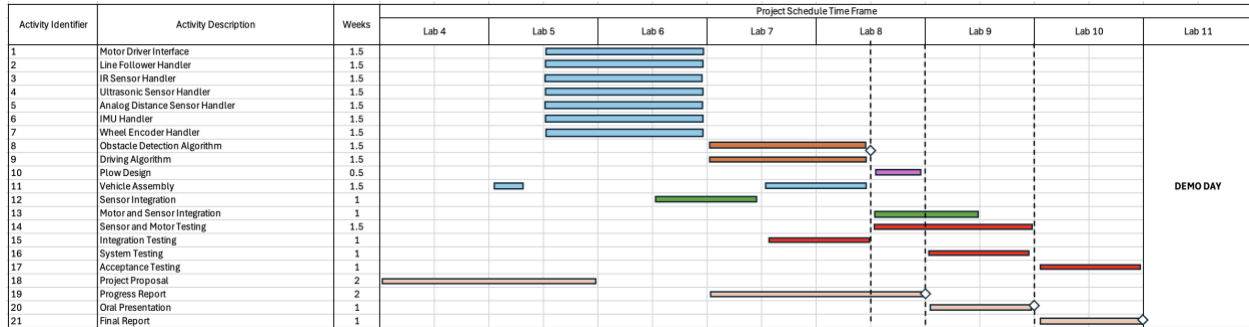


Figure 3: Updated Gantt chart for project timelines for each activity

4 Cost

In terms of project timeline, it is vital to understand the associated costs for the development of each feature including developer costs and the cost of the kit itself. The table below describes the cost breakdown of each activity in terms of the number of developers and the hours of development necessary for each.

Provided below is the original Planned Value (PV) analysis of the project made during the project proposal. The updated planned value analysis is provided in Section 6, Progress Update.

Table 8: Cost Baseline Figures

Item	Number of Developers	Hours of Development	Cost (\$)	Notes
Course Kit	N/A	N/A	500	Includes robot chassis, AA batteries, motor driver, spacers, Arduino due, Arduino nano every, 3.3V buck regulator, various sensors, wiring, resistors, and misc.
Motor Driver Interface Development	1	4	200	
Line Follower Handler Development	1	4	200	
IR Sensor Handler Development	1	4	200	
Ultrasonic Sensor Handler Development	1	4	200	
Analog Distance Sensor Handler Development	1	4	200	
Wheel Encoder Handler Development	1	4	200	

Item	Number of Developers	Hours of Development	Cost (\$)	Notes
Mapping Algorithm Development	2	1	100	The mapping algorithm was considered for an hour before disbanding this task.
Obstacle Detection Algorithm Development	2	8	800	The following algorithms are the “hardest” part of the project and thus, is all hands-on deck in their development. Their also fundamental to the design
Driving Algorithm Development	2	8	800	The following algorithms are the “hardest” part of the project and thus, is all hands-on deck in their development. Their also fundamental to the design
Plow Designing	1	2	100	
Vehicle Assembly	1	2	100	
Sensor Integration	1	4	200	
Mapping and Driving Integration	2	1	100	As part of the mapping algorithm, this activity was disbanded, but 1 hour was spent developing.
Motor and Sensor Integration	1	4	200	
Sensor and Motor Testing	1	4	200	
Integration Testing	1	4	200	
System Testing	1	4	200	
Acceptance Testing	1	4	200	
Project Proposal	4	8	1600	
Progress Report	4	8	1600	
Oral Presentation	4	4	800	
Final Report	4	4	800	
Total	N/A	184	9,700	

Note, developers are charged \$50/hour/developer. Evidently, the cost baseline is \$9,700

5 Human Resources

The following section describes the overall human resource aspects of the project, namely the distribution of tasks amongst the team. In terms of progress, it is vital to note that the team has abided by the responsibility matrix in terms of how the system is developed and task division.

5.1 Responsibility Assignment Matrix

The following table describes the responsibility distribution of each activity for each of the members. Note, responsible (R) is the team member responsible for development and testing, accountable (A) is the team member who must review the work, consulted (C) is the team member who is consulted for the

implementation and design, informed (I) is the team member who has a general idea of the goals and development during the activity.

Table 9.1: RACI Legend

RACI	
R	Responsible
A	Accountable
C	Consulted
I	Informed

Table 9.2: RACI Matrix

Activity	Project Leader Karran Dhillon	Collin Dang	Alex Tempel	Qasim Hamid
1. Motor Driver Interface	R	A	I	C
2. Line Follower Handler	A	C	R	I
3. IR Sensor Handler	C	R	I	A
4. Ultrasonic Sensor Handler	A	I	C	R
5. Analog Distance Sensor Handler	C	A	R	I
6. IMU Handler	A	C	R	I
7. Wheel Encoder Handler	R	I	A	C
8. Obstacle Detection Algorithm	R	I	A	C
9. Driving Algorithm	R	C	I	R
10. Plow Design	I	R	A	C
11. Vehicle Assembly	I	R	A	C
12. Sensor Integration	R	A	C	I
13. Motor and Sensor Integration	I	A	C	R
14. Sensor Testing	A	C	R	I
15. Integration Testing	I	R	C	A
16. System Testing	R	I	C	A
17. Acceptance Testing	C	A	I	R
18. Project Proposal	A	R	R	R
19. Progress Report	A	R	R	R
20. Project Presentation	R	C	I	A
21. Final Report	A	R	R	R

5.2 Contribution of Each Member

The team stuck to the RACI in Table 9.2 very strictly. To summarize the contributions of each member, they are provided below.

Karran, the team lead, implemented the motor driver, wheel encoder handlers and the core driving (RTOS) and obstacle detection algorithms. As well, Karran integrated the sensors on the chassis while consulting with the team on the project proposal, progress report, and final report. Ultimately, the work was iterative with constant communication with the team on the interface and implementation to help integrate the system and obtain an optimal solution.

Alex implemented the sensor handlers for the Analog Distance Sensor, Line Follower, and the IMU. To increase the accuracy of these sensors, he used linear interpolation and rolling averaging. He also created the unit test code, which is used to verify that sensors are functioning properly. He also researched how to run FreeRTOS on the Arduino Due and how to use its API for creating tasks and delays. He also helped test and tweak the detection and driving algorithms. He dialed in the correct detection thresholds for the sensors, so that the snowplow would avoid obstacles but not turn from false-positives or too early. For the driving algorithm he dialed in the distance to reverse after detecting an obstacle or boundary so that it would avoid the obstacle, but not lose too many cubes, he also implemented the random turning algorithm so that the snowplow would cover the area more completely.

Collin, implemented the IR distance sensors, time of flight sensors, designed and built the plow and the robot, and helped with integration testing. Although the IR sensors were not implemented in the final design of the robot due to unreliable hardware, he made sure that the code was up to date with the standards that were required. If the team needed to add the IR sensors to the robot, they would be easily integrated into the RTOS system. Similarly, the time-of-flight sensors were not used either. This is because of some integration and timing issues with the code. Although they worked perfectly in an independent simulated environment, timing issues arose when trying to integrate it with RTOS. He designed and built the plow using cardboard, hot glue, and clear plastic. A prototype was made, and he consulted with the team to update and improve the plow when any issues arose. Collin also did most of the assembly of the robot. He was the one to make sure all the hardware and the structure of the robot were firmly in place. He made sure to consult with the team to make sure that the placements of all the sensors met the requirements to achieve the desired goal and installed the plow so that it did not affect any of the sensors surrounding it.

Qasim implemented the logic for the main ultrasonic sensors and supported the team with the driving algorithm and integration testing of the robot. He initially generated working code for the ultrasonic sensors by testing and using an Arduino Uno. He then worked with the team in the following lab sessions when doing integration testing to confirm the performance of the sensor were replicated on the main Arduino board. As the project evolved, he would keep up with problems faced by the group in the integration testing phase and constantly looked to provide solutions to issues, as the rest of his group members did as well. His experience with RTOS was valuable when deciding the priorities of tasks, and the message passing methods used between threads (semaphores). He had overseen recording values early on for the motor driver in the guided labs, which meant he was familiar with the performance of their specific motor driver. He was helpful in setting the initial values of the motor driver to have basic driving controls of the robot. Since he was involved with most of the project's areas, he was trusted with communicating the challenges the team faced in the final presentation.

6 Progress Update

Substantial progress has been made on the integration of sensors on to the robot (in terms of assembly and software), the use of a Real-Time Operating System (RTOS) for the primary driving algorithm, and integration of the sensor components for the final acceptance testing. The following section describes the project progress in terms of software, design, implementation, and most importantly testing.

In terms of adjustments made to the project proposal and progress report, the mapping algorithm was removed, the Gantt chart has been updated to reflect more up-to-date timelines for tasks to be done and completed, costs adjusted for abandoning the mapping algorithm, and the Responsibility Traceability Matrix (RTM) updated with the actual developers of the activity as adjustments to workload were made.

The mapping algorithm was removed primarily due to complex implementation issues and the timeline of the project. The team had to spend extra time hardening the individual sensor units which overflowed into the time to develop the driving and mapping algorithm. Moreover, the team decided to use freeRTOS which added another task (as part of the driving algorithm task) further delaying timelines and thus, decided on abandoning the mapping algorithm which is more of an enhancement than the core functionality. The team values concrete, tested, and consistent systems over a product with potential bugs and complex logic. The TA, Igor, mentioned that random turns rather than pre-calculated movements achieved better results during past iterations of the project which furthered the case to abandon the driving algorithm.

Overall, as the project begins to wrap up, the overall deliverables, milestones, and success criteria the team wanted to achieve originally are mostly fulfilled. The object was able to stay in bounds, generally avoid obstacles in its path, and most importantly, push snow outside the arena. Further information on the results of the integration testing that ultimately validated the functional and non-functional requirements are explored later in this section.

6.1 System Architecture

The following figure describes the overall architecture of the system in terms of sensors and hardware used.

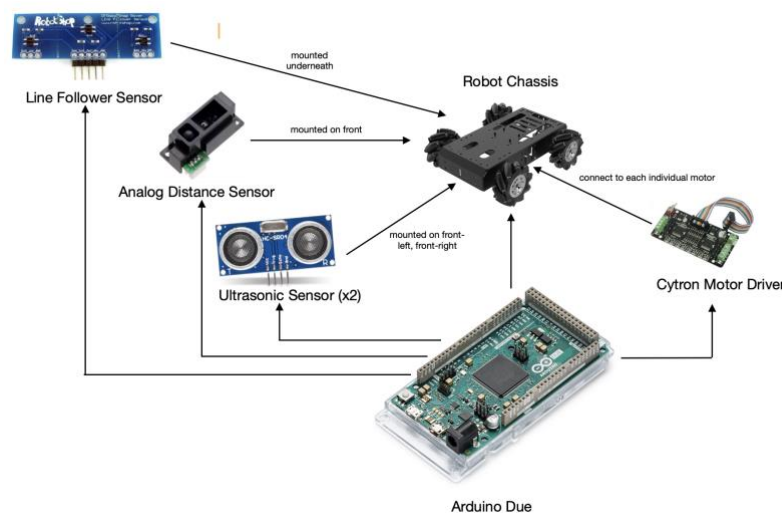


Figure 4: System architecture drawing of robot

The robot uses three main sensors: the Analog Distance Sensor (ADS), ultrasonic sensor, and line follower. In terms of forward detection, the ADS and ultrasonic are used to sense where the object is and how far. If the object is within 10cm, the object is deemed as an “obstacle,” and thus corrective measures must be taken by the primary driving algorithm. The use of 3 total forward sensors (2 ultrasonic, 1 ADS) ensures obstacles on the left, right, and middle are found while minimizing blind spots. This will be further explained in the state chart diagram below. The line follower is within 3cm of the floor to ensure atmospheric light does not interfere with the detection of a black line (i.e. the boundaries of the box).

Moreover, a picture of the assembled robot is provided below to capture a better visual understanding of how the team assembled the system. Note, the Cytron motor driver is not shown, but is bottom mounted under the upper frame of the chassis.

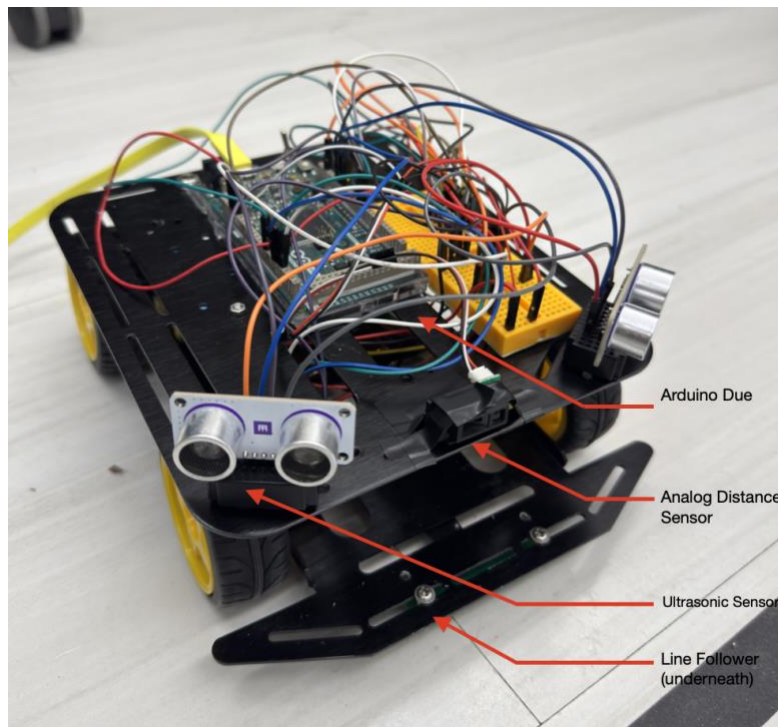


Figure 5: Picture of the assembled robot

In terms of software architecture, the team elected for an Object-Oriented Programming (OOP) approach to help with modularity and hiding complex logic under objects. Each of the sensors get their own class with functionalities specific to the sensor’s purpose. This makes the main driving algorithms, which are represented as tasks in RTOS, easy to implement and builds a solid software architecture. Provided below is a class diagram to further emphasize the importance and use of OOP in the design of the system.

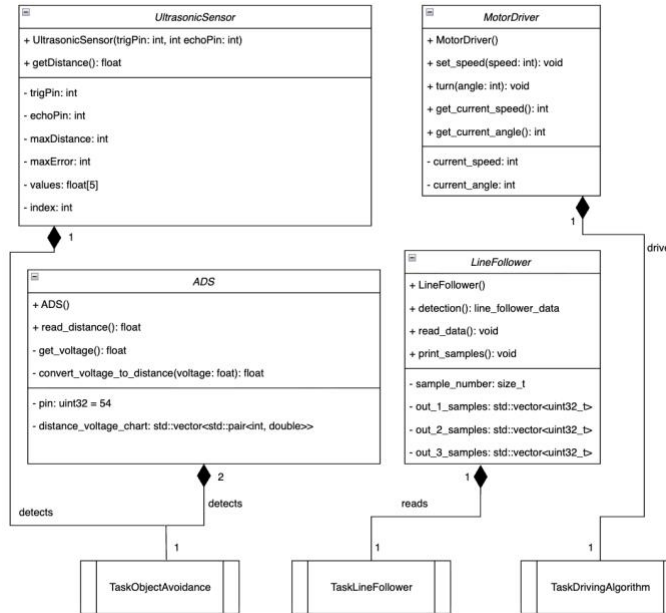


Figure 6: Class diagram describing architecture of the system

6.2 Sequence Diagram

The follow section describes the sequencing of activities and actions performed by the robot during operation for its dynamic obstacle detection and line following.

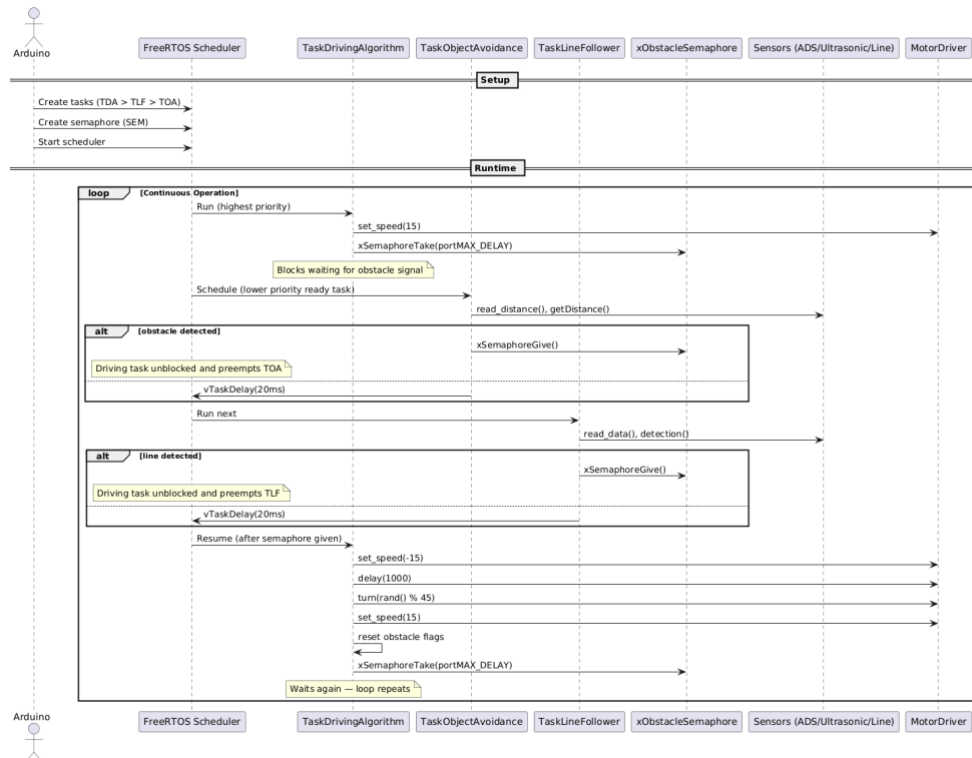


Figure 7: Sequence diagram describing the dynamic behaviour of the robot

In terms of sequencing, the robot is split into three separate processes: the driving algorithm, obstacle detection, and line detection. The sequence diagram depicts that the system is by default always running the driving algorithm (which encapsulates both of the other processes) as it is the lowest priority and thus first process in the sequence diagram. Since the Arduino being used is single core, the robot signals the other threads to check for an object and line. If an object or line is detected, a semaphore is given and the bottom part of the sequence is run, where the robot will turn 90 degrees. This sequence continuously loops.

6.3 Watchdog Timer Implementation

Watchdogs are vital in real-time embedded systems for ensuring the system can detect and recover from malfunctions. They work by regularly resetting a timer before it expires. If the expiry is hit before its cleared, the system is deemed as malfunctioned and corrective action is taken.

FreeRTOS has built-in functionality for watchdogs in terms of being able to create, reset, and check the watchdog timer. The team used the `watchdogEnable()` and `watchdogReset()` functions in the tasks to setup and use it. The following screenshot outlines its implementation.

```

81 void TaskObjectAvoidance(void *pvParameters)
82 {
83     Serial.println("Task Object Avoidance");
84     (void) pvParameters;
85     for (;;)
86     {
87         // calculate distances
88         float ads_distance = analog_digital_sensor.read_distance() < 25 ? analog_digital_sensor.read_distance() : 0;
89         // Serial.print("ADS distance: "); Serial.println(ads_distance);
90         float left_ultrasonic_distance = ultrasonic_left.getDistance() > 0 ? ultrasonic_left.getDistance() : 0;
91         // Serial.print("Left Ultrasonic distance: "); Serial.println(left_ultrasonic_distance);
92         float right_ultrasonic_distance = ultrasonic_right.getDistance() > 0 ? ultrasonic_right.getDistance() : 0;
93         // Serial.print("Right Ultrasonic distance: "); Serial.println(right_ultrasonic_distance);
94
95         if (ads_distance < 5 && ads_distance > 1) // object within 5cm (1cm minimum)
96         {
97             Serial.println("ADS Sensor: Object Avoidance: Object detected!");
98             ads_obstacle_detected = true;
99             xSemaphoreGive(xObstacleSemaphore); // signal driving task to rotate
100         } else if (left_ultrasonic_distance < 5 && left_ultrasonic_distance > 1) {
101             Serial.println("Left Ultrasonic: Object Avoidance: Object detected!");
102             left_ult_obstacle_detected = true;
103             xSemaphoreGive(xObstacleSemaphore);
104         } else if (right_ultrasonic_distance < 5 && right_ultrasonic_distance > 1) {
105             Serial.println("Right Ultrasonic: Object Avoidance: Object detected!");
106             right_ult_obstacle_detected = true;
107             xSemaphoreGive(xObstacleSemaphore);
108         }
109         watchdogReset();
110         vTaskDelay(time_to_ticks(20));
111     }
112 }

```

Figure 8: Screenshot of source code for watchdog reset

Evidently, the `TaskObjectAvoidance`, responsible for detecting objects, will reset the watchdog every iteration to indicate it is actively monitoring objects and has not been corrupted. The watchdog expiry was set to 1 second. In the case the watchdog is not reset in time, the Arduino will reset and therefore, restart the execution of the program.

6.4 Testing Update

The following section investigates the results of the tests conducted throughout the labs using the schema defined in the project proposal document [4].

6.4.1 Unit Test Results

The following table describes the updated unit tests results from Section 2.5 with an added column on if the test passed or failed. The original table is taken from the project proposal [4].

Table 10: Updated unit testing results

Activity Code	Test	Pass Condition	Passed or Failed
UT1	Laser range finder minimum response distance	Laser range finder reports an object 50cm away with ± 5 cm of error	PASS
UT2	Laser range finder maximum response distance	Laser range finder reports an object 200cm away with ± 5 cm of error	PASS
UT3	Laser range finder minimum rate of readings	Laser range finder records at least 3 value per second	PASS
UT3	Ultrasonic sensor maximum response distance	Ultrasonic sensor tracks objects 30cm away within ± 2 cm of error	PASS
UT4	Ultrasonic sensor minimum response distance	Ultrasonic sensor tracks objects 5cm away within ± 1 cm of error	PASS
UT5	Ultrasonic sensor rate of readings	Ultrasonic sensor records at least 3 values per second	PASS
UT6	Car minimum movement speed	The car can operate above 5cm/s	PASS
UT7	Line follower on with electric tape	The line follower reports a line when above an electric tape	PASS
UT8	Line follower off above ground	The line follower reports no line when above the ground	PASS
UT9	IR sensor detects object	The IR sensors report an object that is 5 cm away from it within ± 1 cm of error	PASS
UT10	IR sensor reports no object	The IR sensor should not report an item when an object is 100cm away from it	PASS
UT11	Analog Digital sensor close range detection	Analog digital sensor can report items 3cm away from it within ± 1 cm of error	PASS
UT12	Analog digital sensor max range detection	Analog digital sensor reads items 10cm away from it within ± 1 cm of error	PASS

Activity Code	Test	Pass Condition	Passed or Failed
UT13	IMU x,y,z-axis orientation accuracy	IMU accurately reports angle of all axes within 20 degrees of error	PASS
UT14	Encoder accurately reports speed	The encoder can track speed within 3cm/s	PASS
UT15	Plow can move 20 objects concurrently	Pushing the plow with our hand, at least 20 cubes (snow) should be moved at the same time	PASS

In terms of how the unit tests were performed, the team developed a unit test script that validates that the returned data from each unit/sensor performs as expected. If a unit test fails, it asserts and prints the failure. Using preprocessor directives, the developer can dynamically select unit tests for the desired sensor. Unit tests for minimum and maximum thresholds check for the -1 return when the radar cannot find it (i.e., the object is out of range). Each unit (sensor) has its own respective tests. In terms of exceeding the defined success criteria, measurements were within a closer deviation than expected, and sensors were able to do more readings than expected. Take, for example, the ultrasonic sensor that provides at least three readings per second. The team achieved significantly higher rates, prompting them to use it on the robot. Below are screenshots of the unit tests running and the associated code.

```

1  /**
2  *
3  * This file describes the line follower sensor implementation
4  *
5  * Author: Alex Tempel
6  *
7  * Copyright 2025-Infinately-Hot
8  */
9
10 #include "AnalogDistanceSensor.hpp"
11 #include "IMU.hpp"
12 #include "Ultrasonic.hpp"
13 #include "WheelEncoder.hpp"
14 #include "ArduinoPins.hpp"
15 #include "LineFollower.hpp"
16 #include "unittest.hpp"
17
18 bool analog_distance_sensor_test() {
19     ADS ads(ADS_PIN);
20     auto data = ads.read_distance();
21     return data != -1.0; // It'll return -1.0 if it fails
22 }
23
24 bool imu_test() {
25     IMU imu;
26     bool passed = imu.start();
27     if (!passed) {
28         return false;
29     }
30     imu.read_sensors();
31     imu.get_angle();
32     imu.get_acceleration();
33     imu.get_mag();
34     imu.get_gyro();
35     return true;
36 }
37
38 bool wheel_encoder_test() {
39     WHEEL_ENCODER wheel_encoder;
40     wheel_encoder.read_speed();
41     return true;
42 }
43
44 bool line_follower_test() {
45     LineFollower line_follower(out_1_pin, out_2_pin, out_3_pin);
46     line_follower.read_data();
47     line_follower.detection();
48     return true;
49 }
50

```

Figure 9.1: Screenshot of unit test code, part 1

```

51 bool ultrasonic_test() {
52   UltrasonicSensor ultrasonic(ULTRASONIC_LEFT_TRIG_PIN, ULTRASONIC_LEFT_ECHO_PIN);
53   auto data = ultrasonic.getDistance();
54   return data != -1;
55 }
56
57 bool unittests() {
58   bool all_passed = true;
59   #ifdef ANALOG_DISTANCE_SENSOR
60   if (!analog_distance_sensor_test()) {
61     all_passed = false;
62     Serial.println("Analog Distance Sensor Test failed");
63   }
64   #endif
65   #ifdef IMU_SENSOR
66   if (!imu_test()) {
67     all_passed = false;
68     Serial.println("IMU Test failed");
69   }
70   #endif
71   #ifdef WHEEL_ENCODER_SENSOR
72   if (!wheel_encoder_test()) {
73     all_passed = false;
74     Serial.println("Wheel Encoder Test failed");
75   }
76   #endif
77   #ifdef LINE_FOLLOWER
78   if (!line_follower_test()) {
79     all_passed = false;
80     Serial.println("Line Follower Test failed");
81   }
82   #endif
83   #ifdef ULTRASONIC_SENSOR
84   if (!ultrasonic_test()) {
85     all_passed = false;
86     Serial.println("Ultrasonic Test failed");
87   }
88   #endif
89   if (all_passed) {
90     Serial.println("All tests passed");
91   } else {
92     Serial.println("Some tests failed");
93   }
94   return all_passed;
95 }

```

Figure 9.2: Screenshot of unit test code, part 2

```

main.ino
  AnalogDistanceSensor.cpp  IMU.cpp  IMU.h  CarMovement.cpp  CarMovement.h
63   Serial.println("Analog Distance Sensor Test failed");
64   }
65   if (!imu_test()) {
66     all_passed = false;
67     Serial.println("IMU Test failed");
68   }
69   if (!wheel_encoder_test()) {
70     all_passed = false;
71     Serial.println("Wheel Encoder Test failed");
72   }
73   if (!line_follower_test()) {
74     all_passed = false;
75     Serial.println("Line Follower Test failed");
76   }
77   if (!ultrasonic_test()) {
78     all_passed = false;
79     Serial.println("Ultrasonic Test failed");
80   }
81   // if (!v5311x_test()) {
82   //   all_passed = false;
83   //   Serial.println("Tof Sensor Test failed");
84   // }
85   // if (!vna330_test()) {
86   //   all_passed = false;
87   //   Serial.println("");
88   // }
89   if (all_passed) {
90     Serial.println("All tests passed");
91   } else {
92     Serial.println("Some tests failed");
93   }
94   return all_passed;
95 }

```

Output: Serial Monitor X

Message (Enter to send message to 'Arduino Due (Programming Port)' on 'COM5')

All tests passed

Figure 9.3: Screenshot of unit test code with output, part 3

6.4.2 System Integration Testing Results

The following table presents the integration test results from Section 2.5, with an additional column indicating whether each test passed or failed. The original table is taken from the project proposal document [4]. Integration testing was performed during labs 9 and 10, and an additional customer acceptance testing has been added to highlight the results during the demo lab.

Table 11: Integration and customer testing results for the system

Activity Code	Test	Pass Condition	Pass or Failed
IT1	Ultrasonic sensor interfacing with a moving robot	Ultrasonic sensor updates the proximity of the robot to an object while moving towards it, sending a signal when 10cm away, within ± 2 cm of error	PASS
IT2	Robot can follow a line with the line follower	The robot can move along the outline of a rectangle, staying within the rectangle's bounds. For a 2-minute trial, it should not go out of bounds at any point.	PASS
IT3	Vehicle can traverse the area of the arena within 3 minutes	Assuming no objects in the way, the vehicle should navigate the area of the arena within 3 minutes with all components attached	PASS
CT1	Customer testing during demo day to validate snow removal in test arena within 5 minutes.	The robot must clear the snow in the 2.5m x 2.5m area, including static and moving obstacles, within 5 minutes at a maximum speed of 30cm/s.	FAIL

To validate the integration testing, the team had to use real-time testing and trial-and-error. Each sensor (ultrasonic, ADS, and line follower) was tested together in real operating conditions rather than in isolation. During integration testing, the system not only met the defined success criteria described in the table above but also exceeded expectations. IT1, the ultrasonic sensor integration, consistently detected objects within ± 2 cm of the specified tolerance. In several trials, it remained within 1cm and detected the distance relatively quickly when an object was suddenly placed on its path, demonstrating its sensitivity and low latency. IT2, line follower integration: the robot reliably followed the rectangular path and did not cross the boundary at all, showcasing its ability to exceed the limit of going out of bounds only once during a 2-minute trial. IT3, complete system arena traversal: due to the randomness of motor turning when an obstacle or line is detected, it consistently navigated through the entire arena. It did so in less than 3 minutes, demonstrating that it exceeded the defined success criteria.

Customer testing was performed during lab 11, the demo day. The original non-functional requirement the team sought to achieve was to clear 95% of the snow in the demo arena – the snowplow cleared ~60%

in the 5-minute time limit. Evidently, the customer testing failed under the system's defined constraints. It also softly hit an obstacle three times. Nevertheless, it did prove parts of the system fully operational: the ability to stay within bounds for the entire five minutes, maximize wooden cubes in the plow, and avoid obstacles when facing its edge rather than a flat face.

6.4.3 Control Chart Results

The control charts below provide evidence of how well the system works, where it needs to get better, and how the team can use test data to keep improving it. The charts demonstrate that while functional requirements are largely met, improvements can enhance reliability and robustness.

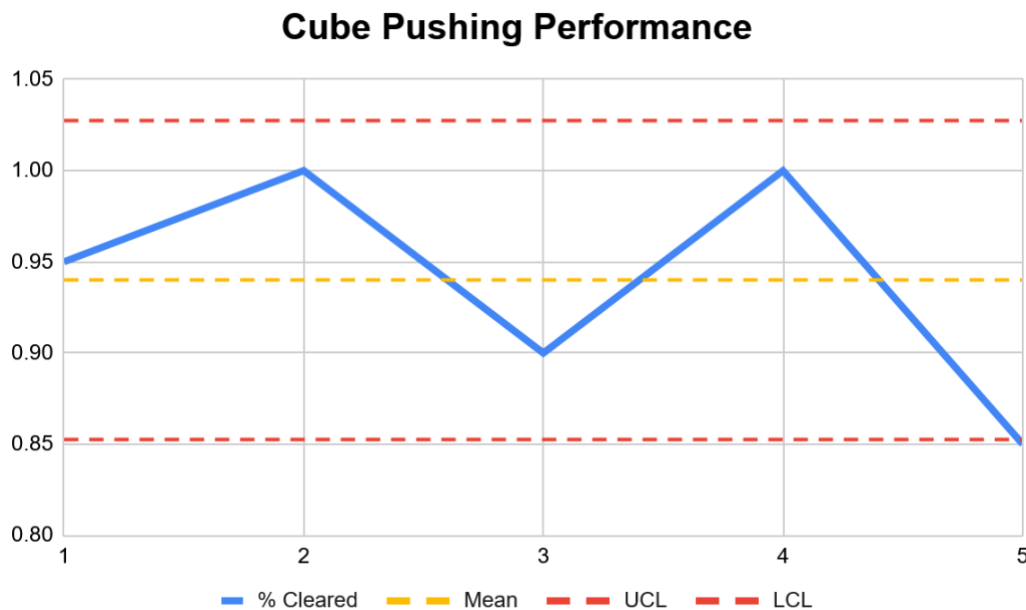


Figure 10.1: Cube Pushing Performance Control Chart

As seen in Figure 10.1, Run #5 fell on/below the LCL, indicating a cause that needs investigation. The team determined that this is due to blocks being missed on certain passes. Blocks that are pushed close to obstacles are generally missed as the robot senses the obstacle before the plow can reach it. Overall, the process is generally capable but shows vulnerability in certain situations where cubes can become stuck against walls or obstacles.

Boundary Detection Performance

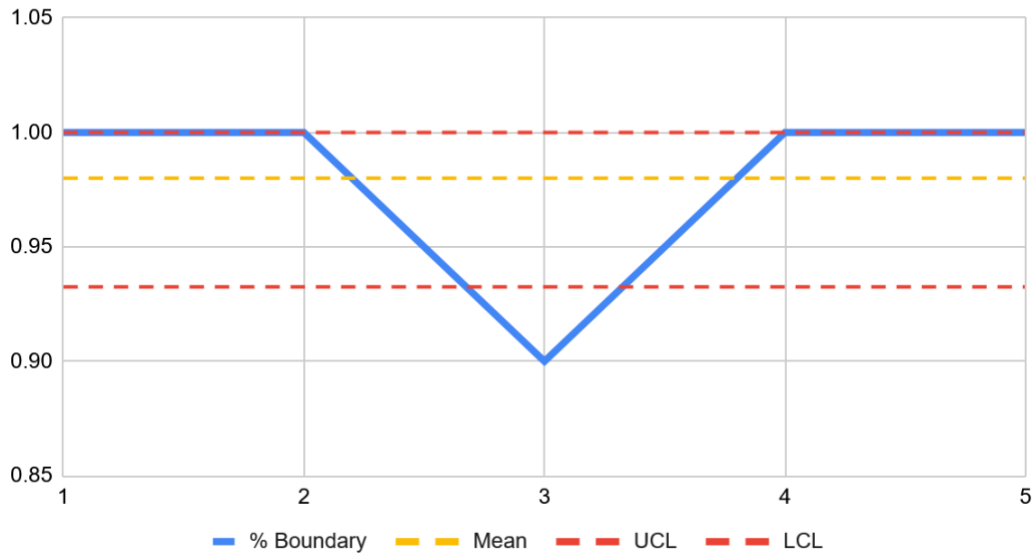


Figure 10.2: Boundary Detection Performance Control Chart

As seen in Figure 10.2, Run #3 fell way below the LCL, indicating a cause that needs immediate investigation. The team determined that this is due to the boundary being dirty. The sensor is typically so accurate that a one small error causes such a dramatic jump in the data. Overall, the process is very reliable and only shows vulnerability in situations outside of the robot's control.

Obstacle Avoidance Performance

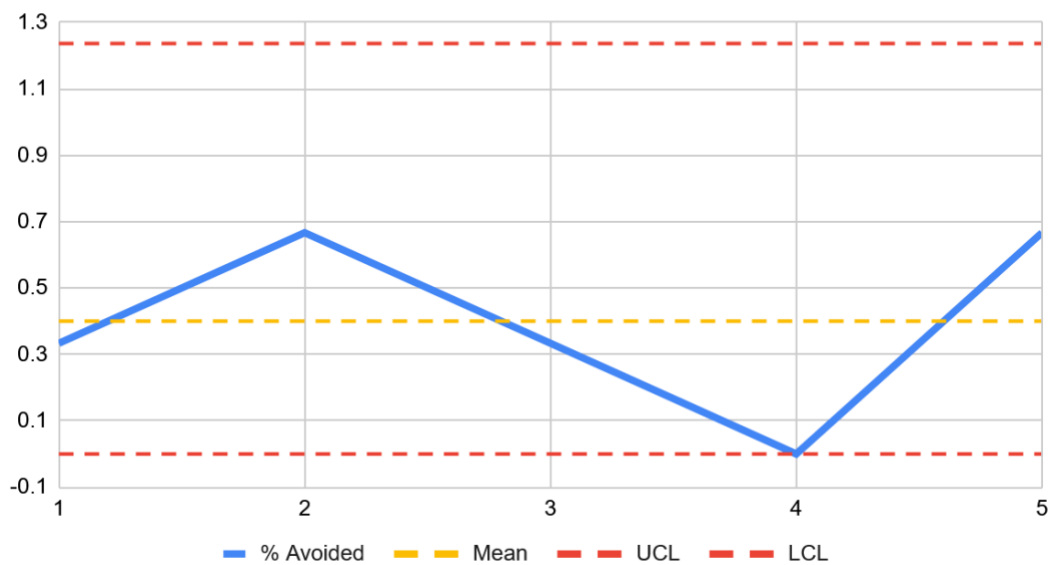


Figure 10.3: Obstacle Avoidance Performance Control Chart

As seen in Figure 10.3, Run #4 fell on the LCL, indicating a cause that needs investigation. We determined that this is due to blind spots between the sensors. Alternating high-low performance suggests variation in the testing. Overall, the performance of the robot depends heavily on obstacles placement; the system handles frontal obstacles well but struggles with lateral obstacles where the signals are not reflected well.

6.5 State chart

The state chart below describes the overall schema for how the system behaves in time and in the presence of obstacles. Note that the system is modelled using tasks; this is therefore not implemented directly but is described in an equivalent state chart.

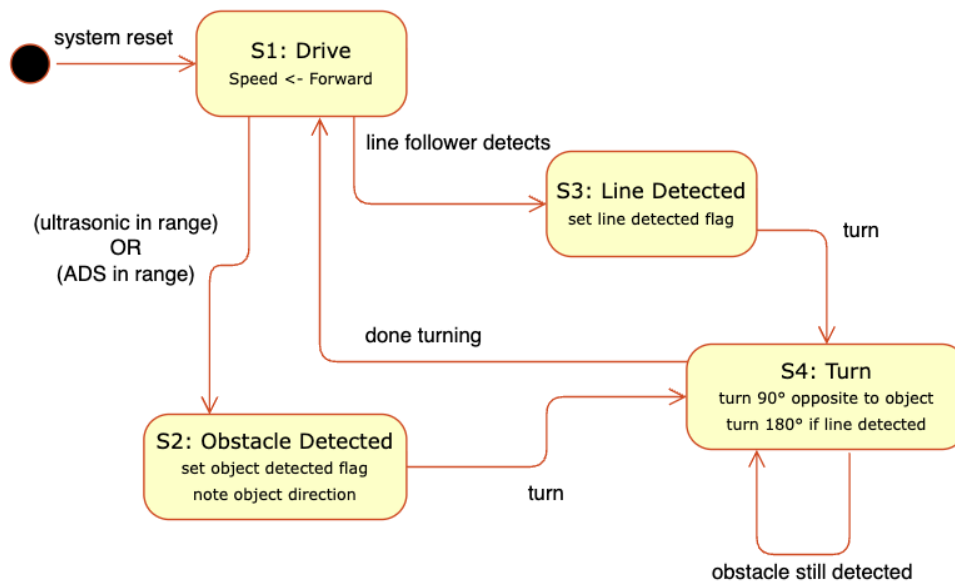


Figure 11: State chart describing driving behaviour

It is clear that the state chart's behaviour is intuitive: maintain a steady forward drive, but if an obstacle or a line is detected (hitting the boundary), the robot must turn until either the object is no longer in range or the line is no longer detected. It is an iterative process that will turn and then re-evaluate the need to continue turning if an object is still in range. This type of state chart is perfect for an RTOS setting, with the foreground, low-priority driving task interrupted by higher-priority obstacle detection and line detection tasks that change the vehicle's state when their respective conditions are met.

A key behaviour the team wanted to maintain was the fact that the algorithm itself was not complex. Adding complexity with more sensors and predefined calculations for turning not only takes more time but also overcomplicates debugging and getting a working core functionality. The team realized that debugging is more complex than implementation, and that minimizing sources of error, including complex algorithms, can make the team more productive and efficient during labs.

6.6 Updated Planned Value Analysis

Provided below is the updated Planned Value analysis that compares the work done (i.e., Estimated Value, EV) as a percentage of the initial cost (PV) to assess the project's schedule and determine whether the team is ahead, behind, or on schedule. The table is taken from the project proposal and section 2 above [4].

Table 12: Updated planned value analysis

Item	Planned Value (\$)	Percentage Complete (%)	Earned Value (\$)	Notes
Course Kit	500	N/A	500	
Motor Driver Interface Development	200	100	200	
Line Follower Handler Development	200	100	200	
IR Sensor Handler Development	200	100	200	
Ultrasonic Sensor Handler Development	200	100	200	
Analog Distance Sensor Handler Development	200	100	200	
Wheel Encoder Handler Development	200	100	200	
Mapping Algorithm Development	100	100	100	An hour spent to disband the topic
Obstacle Detection Algorithm Development	800	100	800	
Driving Algorithm Development	800	100	800	
Plow Designing	100	100	100	
Vehicle Assembly	100	100	100	
Sensor Integration	200	100	200	
Mapping and Driving Integration	100	100	100	An hour spent to disband the topic.
Motor and Sensor Integration	200	100	200	
Sensor and Motor Testing	200	100	200	
Integration Testing	200	100	200	

Item	Planned Value (\$)	Percentage Complete (%)	Earned Value (\$)	Notes
System Testing	200	100	200	
Acceptance Testing	200	100	200	
Project Proposal	1600	100	1600	
Progress Report	1600	100	1600	
Oral Presentation	800	100	800	
Final Report	800	100	800	
Total	9,700	N/A	9,700	

The Planned Value figure given at the start of the project was an estimate of the cost at completion. Evidently, the team has an earned value equal to the planned value meaning the project is finished in the time and money expected.

6.7 Software Development

All of the code discussed throughout the report is available on the GitHub Classroom at: <https://github.com/SYSC4805/project-l3-g15-infinitely-hot>. Please see the README.md in the GitHub repository to fully understand how the code is organized and how to run it.

Code is organized using C++ and .ino files. Each sensor gets its own object with functions for interfacing with the sensor. Take, for example, the motor driver class in the following screenshot:

```

18 // PWM pin defines
19 #define PWM_LEFT 2 // left-side motors (enable/speed pins)
20 #define PWM_RIGHT 3 // right-side motors (enable/speed pins)
21 #define DIR_LEFT 4 // left-side direction (dir)
22 #define DIR_RIGHT 5 // right-side direction (dir)
23
24 // PWM specification
25 // duty cycle is configurable between 0 (0%) - 255 (100%)
26 // at a fixed 1kHz frequency
27
28 class MOTOR_DRIVER
29 {
30 public:
31     MOTOR_DRIVER();
32     void set_speed(int speed);
33     void turn(int angle);
34
35     // getters/setters
36     int get_current_speed() { return current_speed; }
37     int get_current_angle() { return current_angle; }
38
39 private:
40     int current_speed;
41     int current_angle;
42 };

```

Figure 12: Screenshot of motor driver class

By using Object Oriented Programming (OOP), the main program, which is the object detection and driving algorithm, is abstracted away from the implementation of the motor movement. By doing so, the main loop can invoke simple functions to perform actions such as moving forward, backward, or turning left or right. The OOP approach is applied to all sensors to make the main driving and object-detection algorithms

easier to read and abstracted from the implementation. Not only does it help with debugging by making the sensors modular, but it also helps designers develop clean, easy-to-read code.

A video is linked [here](#) showcasing how the code works, with the team voicing over the codebase to explain its purpose and usage.

6.8 RTOS Implementation

Real-Time Operating Systems (RTOS) provide deterministic, pre-emptive task execution for efficient real-time (deadlines) programming. RTOS divides work into tasks with priorities and recurrence frequencies; as a higher-priority task is scheduled to run, it pre-empts (kicks out) the lower-priority task to run the more time-sensitive task. As a rule of thumb, the shorter the deadline, the higher the priority.

The team decided to go with three tasks, each of which is included in the main.ino file. The three tasks are: forward object detection (TaskObjectAvoidance), line follower detection (TaskLineFollower) and the main driving algorithm (TaskDrivingAlgorithm). The object avoidance task, with the lowest priority, has a 200ms periodicity (5Hz) and checks whether an object is within 10cm. The line follower task, with the second-highest priority, runs at a 200ms period and checks whether a black line is beneath it to ensure it stays in bounds. The driving algorithm task, with the highest priority, performs forward movement, backups, and turns when an object or line is detected. Intuitively, the driving algorithm is given the highest priority, as its deadline for taking corrective action to avoid the obstacle or prevent going over a line is the shortest. Moreover, because the driving environment can change frequently (e.g., moving obstacles and a fast-moving robot), it is vital to check the sensors for objects and lines regularly. At 5Hz, this is sufficient to detect and quickly take corrective actions while minimizing power consumption. In FreeRTOS, the functions used are: xTaskCreate with each task's priority, vTaskDelay for periodic execution, and watchdogReset for system error correction.

To provide synchronization and coordinate tasks, a key part of an RTOS system, the object avoidance and line follower task will signal a binary semaphore and set a flag if an object or line is detected, as in an Interrupt Service Routine (ISR). The driving algorithm is blocked, waiting to take the semaphore. At this point, it will perform a corrective action: rotate randomly until the object is no longer detected, then resume the path it was following. In terms of how the driving algorithm knows what corrective action to take, before giving the semaphore, those tasks set a static flag that the driving algorithm reads to determine where the obstacle is or whether a line was detected. This functionality exercises freeRTOS's SemaphoreHandle_t, xSemaphoreGive, and xSemaphoreTake with indefinite blocking. Indefinite blocks simplify design and save power instead of a traditional super loop approach.

Provided below is a diagram of the designed Gantt chart for deterministic task execution. It outlines why the team chose the given execution times, periodicity, and priority.

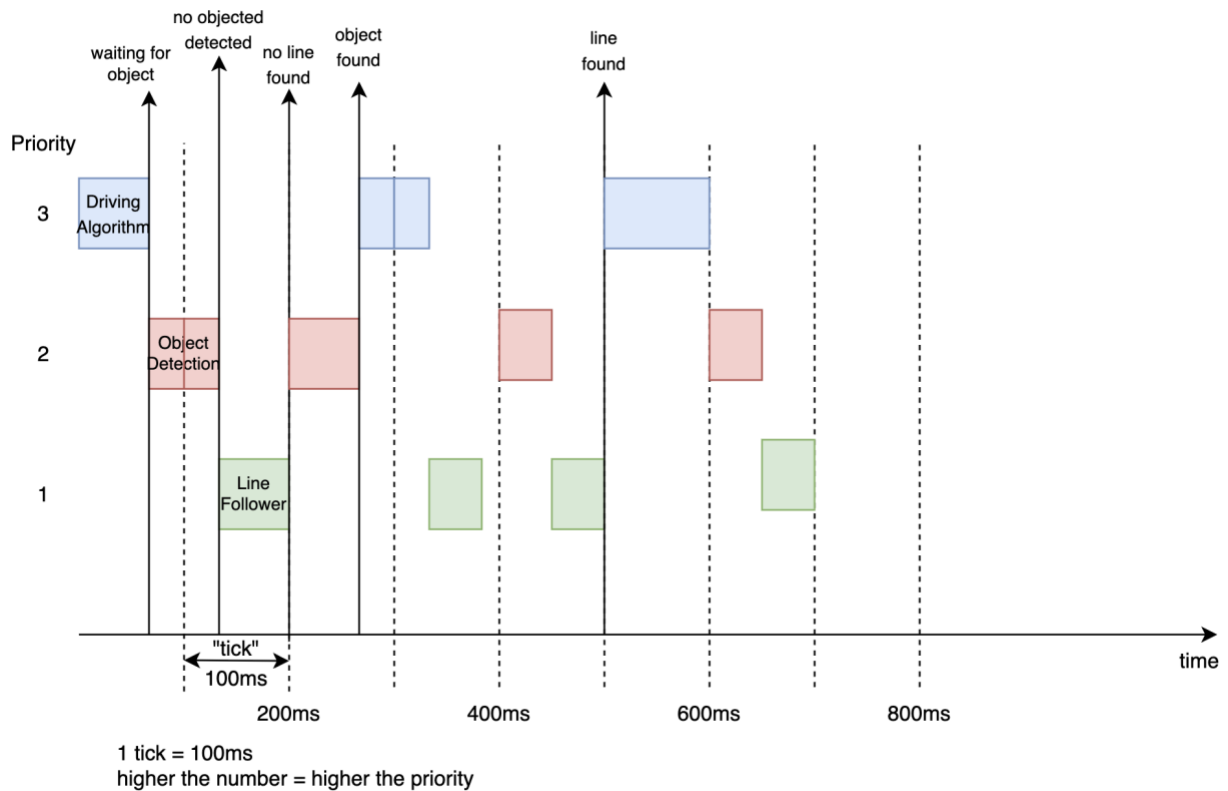


Figure 13: Gantt chart describing task execution

As shown in the figure above, the task periodicity is 200ms; they are rescheduled for execution every 200ms. Note that the actual seconds per tick are 1ms, but are shown as 100ms for easier visualization of the Gantt chart data. Even though the driving algorithm is of highest priority, because it is indefinitely waiting for the semaphore given by the object detection and line follower tasks, it will only be rescheduled for execution after it takes corrective action and needs to wait for the semaphore again. Execution time of each task has been experimentally determined to fill in the figure above. The driving algorithm is shown to take longer when corrective action is needed compared to just waiting for the semaphore.

7 References

- [1] B. Rose, "Snow plow nearly runs over Minnesota Vikings players (GIF)," *FanSided*, Dec. 08, 2013. <https://fansided.com/2013/12/08/snow-plow-nearly-runs-minnesota-vikings-players-gif/> (accessed Oct. 01, 2025).
- [2] *Brightspace.carleton.ca*, "Project Description: Autonomous Snowplow", 2025. <https://brightspace.carleton.ca/d2l/le/content/372623/viewContent/4450646/View> (accessed Oct. 01, 2025).
- [3] *Brightspace.carleton.ca*, "Project Report: Autonomous Snowplow", 2025. <https://brightspace.carleton.ca/d2l/le/content/372623/viewContent/4450655/View> (accessed Nov. 09, 2025).
- [4] K. Dhillon, A. Tempel, C. Dang, and Q. Hamid, "SYSC 4805: Autonomous Snowplow Project Proposal," Oct. 2025.
- [5] K. Dhillon, A. Tempel, C. Dang, and Q. Hamid, "SYSC 4805: Autonomous Snowplow Progress Report," Oct. 2025.